



School Uniform & Supplies Marketplace

Software Architecture Specification Document

Document Version:	1.0
Classification:	Confidential
Prepared For:	Investors · Development · Product · Database Teams
Date:	June 14, 2026
Phase:	Phase 1 — Marketplace MVP

The School Uniform & Supplies Marketplace is a technology-driven, multi-sided marketplace platform that digitises the procurement of school uniforms and educational supplies for schools, parents, and students across India. The platform bridges the gap between uniform vendors and educational institutions by providing a structured, transparent, and scalable digital commerce layer.

1.1 Platform Model

The platform operates as a pure marketplace intermediary. It does not warehouse, purchase, or physically handle any goods. All inventory ownership, stock management, and fulfilment responsibility rests with registered Vendors. The platform earns revenue through a commission-based model applied on each completed transaction.



1.2 Stakeholders

Stakeholder	Role
Vendors	Third-party suppliers who list, price, and fulfil school uniform and supply products.
Schools	Educational institutions that curate and approve uniform product lists per class and gender.
Parents	End consumers who discover, select, and purchase approved school products.
Students	Beneficiaries of purchased products, linked to a school and class.
Admins	Platform operators who onboard vendors/schools, approve products, and manage commissions.

1.3 Business Goals

- Digitise uniform procurement for 1,000+ schools across India within 24 months.
- Provide vendors with a structured digital storefront and real-time order visibility.
- Enable schools to maintain a curated, class-wise, gender-wise product catalogue.
- Generate platform revenue through a transparent commission model per order.
- Deliver a seamless, mobile-first ordering experience for parents.
- Build a scalable, API-first architecture capable of supporting 100,000+ concurrent users.

Note: Phase 1 focuses exclusively on Product Listing, Variant Management, Stock Tracking, and School Product Mapping. Warehouse management, returns, and coupon modules are explicitly deferred to future phases.

Document Version: 1.0
Classification: Confidential
Prepared For: Investors · Development · Product · Database Teams
Date: June 14, 2026
Phase: Phase 1 — Marketplace MVP

Section 2 — System Overview

2. System Overview

The platform is composed of five primary actor-types interacting through a set of well-defined modules. Each actor has specific responsibilities, permissions, and data visibility scopes.

Vendor	Responsibilities
	• Self-register and submit KYC documents (GSTIN, PAN).
	• List products across approved categories.
	• Define product variants (size, colour, MRP, selling price).
	• Manage stock quantities per variant.
	• Receive and fulfil orders.
	• View earnings and commission deductions via dashboard.
School	Responsibilities
	• Onboarded by platform admin.
	• Browse platform-approved product catalogue.
	• Approve or reject products for each class and gender.
	• Set academic year and mandatory/optional flags per mapping.
	• View order volumes from parent community.
Parent	Responsibilities
	• Register and link to a school and student.
	• Browse school-curated product catalogue.
	• Order products across multiple vendors in one cart.
	• Track order and payment status.
	• View order history.
Student	Responsibilities
	• Profile linked to parent, school, and class.
	• Used to filter the correct uniform list (class + gender).
	• Multiple students per parent supported.
Admin	Responsibilities

	• Onboard and verify vendors (KYC review).
	• Approve or reject product listings.
	• Onboard schools.
	• Manage categories and platform settings.
	• Monitor orders, commissions, and analytics.
	• Configure commission rates per vendor.

2.1 High-Level Workflow

Step	Stage	Description
1	Vendor Registration	Vendor submits registration → Admin reviews KYC → Account activated
2	Product Listing	Vendor creates product + variants → Admin approves product
3	School Onboarding	Admin creates school profile → School admin credentials issued
4	Product Mapping	School browses approved products → Maps products per class/gender/year
5	Parent Registration	Parent registers → Links to school → Adds student profiles
6	Ordering	Parent views school product list → Adds to cart → Completes payment
7	Fulfilment	Order routed to vendor → Vendor fulfils → Stock auto-decremented
8	Commission	Platform deducts commission → Net amount released to vendor

Section 3 — Product Listing Architecture

3. Product Listing Architecture

A fundamental architectural decision in this platform is the strict separation of Products and Product Variants into two distinct entities. This is not an implementation convenience — it is a core data modelling principle that unlocks scalability, correctness, and maintainability.

3.1 Why Products and Variants Must Be Separate

Consider a single product: White School Shirt. This product exists in multiple sizes (S, M, L, XL) and potentially multiple colours. Each combination of attributes constitutes a distinct purchasable unit (SKU) with its own:

- Stock quantity
- Selling price and MRP
- Barcode / SKU code
- Low stock threshold
- Active/inactive status

Product Level	Variant Level (SKU)
product_name: White School Shirt category: Shirt material: Cotton Blend gender_type: Boys approval_status: Approved	SKU-001: Size S MRP █350 Stock 20 SKU-002: Size M MRP █370 Stock 50 SKU-003: Size L MRP █390 Stock 30 SKU-004: Size XL MRP █410 Stock 15

3.2 Scalability Benefits

Benefit	Explanation
Single Source of Truth	All non-variant product metadata (name, description, images, category, approval) is stored once at the product level. No duplication across 10+ variants.
Independent Pricing	Variant-level pricing allows size-based price differentials without schema changes.
Granular Stock Control	Stock is tracked per SKU. An XL-size going out of stock does not affect Size S availability.
Admin Approval Efficiency	Product approval operates at the product level. A single approval unlocks all its variants for school mapping.
School Mapping Precision	Schools can map at the product level. Parents can then select their required variant (size) at order time.
Future-Proof	New variant attributes (e.g., colour, fabric type) can be added to the variants table without restructuring the entire schema.

Section 4 — Database Design

4. Database Design

All tables use UUIDs as primary keys to ensure global uniqueness, horizontal scalability, and safe API exposure without sequential ID enumeration. The schema is designed for MySQL 8.0+ / PostgreSQL 14+ with Laravel Eloquent ORM compatibility.

Table: vendors

Purpose: Stores all registered vendor/supplier profiles including KYC and commission details.

Field	Type	Constraints	Description
vendor_id	UUID	PK, NOT NULL	Universally unique identifier for each vendor.
business_name	VARCHAR(150)	NOT NULL	Registered business or trade name of the vendor.
owner_name	VARCHAR(100)	NOT NULL	Full name of the primary owner/contact person.
email	VARCHAR(120)	UNIQUE, NOT NULL	Primary email used for login and communication.
phone	VARCHAR(15)	UNIQUE, NOT NULL	Mobile number; used for OTP-based authentication.
gst_in	VARCHAR(15)	UNIQUE, NULLABLE	GST Identification Number for tax compliance.
pan_number	VARCHAR(10)	UNIQUE, NULLABLE	PAN for TDS deduction and income tax reporting.
commission_rate	DECIMAL(5,2)	NOT NULL, DEFAULT 10	Platform commission percentage applied per order.
status	ENUM	NOT NULL, DEFAULT pending	Values: pending active suspended rejected.
created_at	TIMESTAMP	NOT NULL	Record creation timestamp (auto-set by Laravel).
updated_at	TIMESTAMP	NOT NULL	Last update timestamp (auto-managed by Eloquent).

Table: categories

Purpose: Master list of product categories. Drives category-specific UI behaviour such as size selectors.

Field	Type	Constraints	Description
category_id	UUID	PK, NOT NULL	Unique identifier for the category.
category_name	VARCHAR(80)	UNIQUE, NOT NULL	Display name. Examples: Shirt, Trouser, Notebook, Bag.
category_type	ENUM	NOT NULL	Values: uniform stationery accessory footwear.
requires_size	BOOLEAN	NOT NULL, DEFAULT true	Drives size selector in UI. TRUE for Shirt, FALSE for Pen.
is_active	BOOLEAN	NOT NULL, DEFAULT true	Soft-delete flag. Inactive categories hide from listing.
created_at	TIMESTAMP	NOT NULL	Record creation timestamp.

Table: products

Purpose: Core product entity. Stores shared product metadata. Stock quantities are NEVER stored here.

Field	Type	Constraints	Description
product_id	UUID	PK, NOT NULL	Unique identifier for the product.
vendor_id	UUID	FK → vendors, NOT NULL	Owning vendor. Indexed for vendor-scoped queries.
category_id	UUID	FK → categories, NOT NULL	Product category. Drives UI and filtering.
product_name	VARCHAR(200)	NOT NULL	Full product name. e.g., White School Shirt Boys.
description	TEXT	NULLABLE	Detailed product description, fabric, care instructions.
material	VARCHAR(100)	NULLABLE	Fabric/material type. e.g., 65% Polyester 35% Cotton.
gender_type	ENUM	NOT NULL	Values: boys girls unisex.
approval_status	ENUM	NOT NULL, DEFAULT pending	Values: pending approved rejected.
is_active	BOOLEAN	NOT NULL, DEFAULT true	Vendor-controlled visibility toggle.
created_at	TIMESTAMP	NOT NULL	Record creation timestamp.
updated_at	TIMESTAMP	NOT NULL	Last modification timestamp.

Why stock_qty is NOT stored in the products table: A product such as "White School Shirt" exists in 4 sizes. Each size has a different stock level. Storing stock at the product level would require aggregating across sizes — losing granularity, causing incorrect out-of-stock signals, and making size-level low-stock alerts impossible. Stock belongs exclusively in product_variants.

Table: product_variants

Purpose: THE most critical table. Each row represents a purchasable SKU with its own price, stock, and barcode.

Field	Type	Constraints	Description
variant_id	UUID	PK, NOT NULL	Unique SKU identifier.
product_id	UUID	FK → products, NOT NULL	Parent product. Indexed for fast lookups.
sku	VARCHAR(60)	UNIQUE, NOT NULL	Stock Keeping Unit code. e.g., WSS-BYS-M-WHT.
size	VARCHAR(20)	NULLABLE	Size label: S, M, L, XL, 28, 30 etc. NULL for no-size items.
color	VARCHAR(40)	NULLABLE	Colour name or hex. e.g., White, Navy Blue, #1B2A4A.
mrp	DECIMAL(10,2)	NOT NULL	Maximum Retail Price as per government guidelines.
selling_price	DECIMAL(10,2)	NOT NULL	Actual price charged to parent. Must be ≤ MRP.
stock_qty	INT UNSIGNED	NOT NULL, DEFAULT 0	Current available stock. Decrement on order placement.
low_stock_alert	INT UNSIGNED	NOT NULL, DEFAULT 5	Threshold below which low-stock warning triggers.
barcode	VARCHAR(60)	UNIQUE, NULLABLE	EAN-13 or QR code for physical scanning by vendor.
is_active	BOOLEAN	NOT NULL, DEFAULT true	Inactive variants hidden from parent catalogue.
created_at	TIMESTAMP	NOT NULL	Record creation timestamp.
updated_at	TIMESTAMP	NOT NULL	Auto-updated on stock changes.

Example Data — product_variants (White School Shirt, Boys)

variant_id	sku	size	mrp	selling_price	stock_qty
uuid-v1-...	WSS-BYS-S-WHT	S	■350	■320	20
uuid-v2-...	WSS-BYS-M-WHT	M	■370	■340	50
uuid-v3-...	WSS-BYS-L-WHT	L	■390	■360	30
uuid-v4-...	WSS-BYS-XL-WHT	XL	■410	■380	15

Table: product_images

Purpose: Stores product image metadata. Each image is a separate row to support ordering, primary selection, and CRUD operations.

Field	Type	Constraints	Description
image_id	UUID	PK, NOT NULL	Unique identifier for each image record.
product_id	UUID	FK → products, NOT NULL	Parent product. Indexed for fast image retrieval.
image_path	VARCHAR(500)	NOT NULL	CDN or S3 storage path. e.g., /products/uuid/front.webp.
is_primary	BOOLEAN	NOT NULL, DEFAULT false	Only one row per product should be true. Used for thumbnails.
sort_order	INT	NOT NULL, DEFAULT 0	Display sequence. Lower value = shown first.
created_at	TIMESTAMP	NOT NULL	Upload timestamp.

Why images must NOT be stored as JSON in the products table: Storing image paths as a JSON array (e.g., images: ["a.jpg","b.jpg"]) creates several critical problems: (1) It is impossible to query a single image by ID for deletion or replacement. (2) Sort order and primary-flag logic require application-side JSON parsing instead of a simple SQL ORDER BY. (3) It prevents database-level indexing and referential integrity. (4) CDN cache invalidation for a single image becomes impossible without reading and rewriting the entire JSON blob. A normalised image table solves all these issues.

Table: schools

Purpose: Stores school profiles. Schools are onboarded by platform admins.

Field	Type	Constraints	Description
school_id	UUID	PK, NOT NULL	Unique school identifier.
school_name	VARCHAR(200)	NOT NULL	Full official name of the institution.
address	TEXT	NOT NULL	Complete postal address including city, state, pin.
status	ENUM	NOT NULL	Values: active inactive. Controls parent visibility.
created_at	TIMESTAMP	NOT NULL	Onboarding date.
updated_at	TIMESTAMP	NOT NULL	Last profile update timestamp.

Table: school_product_mappings

Purpose: Maps approved products to schools with class, gender, academic year, and mandatory/optional flags.

Field	Type	Constraints	Description
mapping_id	UUID	PK, NOT NULL	Unique mapping identifier.
school_id	UUID	FK → schools, NOT NULL	The school this mapping belongs to.
product_id	UUID	FK → products, NOT NULL	The platform-approved product being mapped.
class_name	VARCHAR(20)	NOT NULL	Class level. e.g., Class 1, Class 6, KG2.
gender_type	ENUM	NOT NULL	boys girls unisex — filters relevant products.
academic_year	VARCHAR(10)	NOT NULL	e.g., 2024-25. Enables year-over-year catalogue versioning.

Field	Type	Constraints	Description
is_mandatory	BOOLEAN	NOT NULL, DEFAULT true	TRUE = required uniform. FALSE = optional purchase.
school_decision	ENUM	NOT NULL, DEFAULT pending	Values: pending approved rejected.
reviewed_by	UUID	FK → users (school admin)	School admin who reviewed the mapping.
reviewed_at	TIMESTAMP	NULLABLE	Timestamp of school approval/rejection decision.
created_at	TIMESTAMP	NOT NULL	When the mapping was first created.

The mapping workflow is: (1) Admin approves a vendor product at the platform level. (2) The school admin browses the approved product catalogue. (3) The school admin maps relevant products to specific classes and genders. (4) The school sets mandatory/optional flags and academic year. (5) Once `school_decision` = approved, the product becomes visible to parents of that school, class, and gender.

Table: parents

Purpose: Stores parent account profiles. Parents are the end consumers of the platform.

Field	Type	Constraints	Description
parent_id	UUID	PK, NOT NULL	Unique parent identifier.
name	VARCHAR(100)	NOT NULL	Full name of the parent/guardian.
email	VARCHAR(120)	UNIQUE, NULLABLE	Email address. Used for order receipts and login.
phone	VARCHAR(15)	UNIQUE, NOT NULL	Primary identifier for OTP-based authentication.
created_at	TIMESTAMP	NOT NULL	Registration timestamp.
updated_at	TIMESTAMP	NOT NULL	Last profile update.

Table: students

Purpose: Student profiles linked to parents. Used to resolve the correct school-class-gender catalogue.

Field	Type	Constraints	Description
student_id	UUID	PK, NOT NULL	Unique student identifier.
parent_id	UUID	FK → parents, NOT NULL	Owning parent. One parent can have multiple students.
school_id	UUID	FK → schools, NOT NULL	School the student is enrolled in.
class_name	VARCHAR(20)	NOT NULL	Student's current class. e.g., Class 4, Grade 8.
gender	ENUM	NOT NULL	boys girls — drives product catalogue filtering.
created_at	TIMESTAMP	NOT NULL	Profile creation timestamp.

Table: orders

Purpose: Top-level order record created when a parent completes checkout. One order per vendor per checkout session.

Field	Type	Constraints	Description
order_id	UUID	PK, NOT NULL	Unique order identifier.
parent_id	UUID	FK → parents, NOT NULL	Parent who placed the order.
student_id	UUID	FK → students, NOT NULL	Student for whom items are ordered.
school_id	UUID	FK → schools, NOT NULL	School context for the order.
vendor_id	UUID	FK → vendors, NOT NULL	Vendor fulfilling this order.
order_total	DECIMAL(10,2)	NOT NULL	Total payable amount including GST.
payment_status	ENUM	NOT NULL, DEFAULT pending	pending paid failed refunded.
order_status	ENUM	NOT NULL, DEFAULT placed	placed confirmed shipped delivered cancelled.
created_at	TIMESTAMP	NOT NULL	Order placement timestamp.

Field	Type	Constraints	Description
updated_at	TIMESTAMP	NOT NULL	Last status update timestamp.

Table: order_items

Purpose: Line items within an order. Each row is one product variant at a specific quantity.

Field	Type	Constraints	Description
item_id	UUID	PK, NOT NULL	Unique line item identifier.
order_id	UUID	FK → orders, NOT NULL	Parent order. Indexed for order detail retrieval.
variant_id	UUID	FK → product_variants, NOT NULL	Specific SKU ordered.
qty	INT UNSIGNED	NOT NULL, DEFAULT 1	Quantity ordered for this variant.
price	DECIMAL(10,2)	NOT NULL	Selling price at time of order (snapshot).
gst	DECIMAL(10,2)	NOT NULL, DEFAULT 0	GST amount applied on this line item.
subtotal	DECIMAL(10,2)	NOT NULL	Computed: (price + gst) × qty.
created_at	TIMESTAMP	NOT NULL	Line item creation timestamp.

Price and GST values are snapshotted at order time. This ensures historical order accuracy even if vendor prices change post-order. The subtotal is persisted (not computed on read) to prevent floating-point drift and support fast reporting queries.

Section 5 — Entity Relationship Diagram

5. Entity Relationship Diagram (Text Representation)

The following diagram represents all entities and their relationships. PK = Primary Key, FK = Foreign Key, 1:N = one-to-many, N:1 = many-to-one, N:M = many-to-many.

```

vendors 1:N products
product_id PK vendor_id PK
business_name FK vendor_id email / phone FK category_id
commission_rate product_name status gender_type
approval_status 1:N
1:N product_images
product_variants PK image_id
PK variant_id FK product_id FK product_id image_path sku / size / color
is_primary mrp / selling_price sort_order stock_qty
low_stock_alert FK (order_items)
categories PK category_id
category_name order_items requires_size
PK item_id FK order_id FK variant_id
schools 1:N qty / price gst / subtotal PK
school_id school_name N:1 address / status
1:N orders
PK order_id FK
parent_id school_product_mappings FK student_id FK
school_id PK mapping_id FK vendor_id FK school_id order_total FK product_id
payment_status class_name order_status academic_year
is_mandatory N:1 school_decision
parents
1:N students PK
parent_id name / email / phone PK student_id
FK parent_id FK school_id class_name gender

```

Section 6 — Product Lifecycle

6. Product Lifecycle

Step	Stage	Detail
1	Vendor Creates Product	Vendor fills product form (name, category, gender, description, material). Uploads images. Defines at least one variant (size, colour, MRP, selling price, stock qty). Saves as draft.
2	Admin Reviews Product	Platform admin receives notification of pending product. Reviews product name, images, variant pricing, and category alignment. Can request revisions, approve, or reject.
3	Product Approved	approval_status updated to 'approved'. Product becomes visible in the school product browser. All variants inherit parent product approval.
4	School Maps Product	School admin browses approved product catalogue. Selects product for a specific class, gender, and academic year. Sets mandatory or optional flag. Sets school_decision to 'approved'.
5	Parent Views Product	Parent logs in and selects student profile. Platform filters school + class + gender. Approved, mapped products are displayed. Variants (sizes) available for selection.
6	Parent Places Order	Parent selects variant, quantity, adds to cart. Checkout triggers payment flow. On payment success, order record is created in 'placed' status.
7	Stock Auto-Decrement	On successful order: stock_qty of each ordered variant is reduced by qty ordered. If post-deduction stock_qty < low_stock_alert, vendor receives low-stock notification.
8	Vendor Fulfils Order	Vendor receives order notification. Ships product. Updates order_status through: confirmed → shipped → delivered. Commission is calculated and recorded on delivery.

Section 7 — Stock Management Strategy

7. Stock Management Strategy

A common misconception in marketplace platform design is conflating stock tracking with inventory management. These are fundamentally different systems with different complexities and costs.

Dimension	Stock Tracking (THIS PLATFORM)	Inventory Management (NOT REQUIRED)
Who owns stock?	Vendor — platform only tracks the number.	Platform purchases and stores its own stock.
Physical tracking?	No. We track a number (stock_qty).	Yes. Bin locations, shelves, warehouse zones.
Purchase orders?	Not required.	Complex PO → GRN → putaway workflows.
Replenishment logic?	Vendor manages independently.	Automated reorder points and supplier management.
Complexity	Low. Single integer decrement per order.	High. Multi-step workflows and audit trails.
Phase	Phase 1 — MVP.	Phase 4 — Optional future module.

7.1 Stock Status States

State	Condition	UI Behaviour	Example
In Stock	stock_qty > low_stock_alert	Add to cart enabled. Green badge.	stock_qty = 50, alert = 5
Low Stock	0 < stock_qty ≤ low_stock_alert	'Only X left!' warning shown. Vendor notified.	stock_qty = 3, alert = 5
Out of Stock	stock_qty = 0	'Out of Stock' badge. Add to cart disabled.	stock_qty = 0

Stock decrement is performed within a database transaction to prevent overselling in concurrent order scenarios. A SELECT FOR UPDATE lock is applied on the variant row before decrement and commit.

Section 8 — Vendor Dashboard Requirements

8. Vendor Dashboard Requirements

Widget	Data Source (SQL Concept)	Purpose
Total Products	<code>COUNT(*) FROM products WHERE vendor_id = ?</code>	All products regardless of approval status.
Total Variants	<code>COUNT(*) FROM product_variants pv JOIN products p</code>	Sum of all SKUs across all vendor products.
Total Stock Value	<code>SUM(stock_qty * selling_price) FROM product_variants</code>	Current inventory value at selling price.
Low Stock Products	<code>variants WHERE stock_qty > 0 AND stock_qty <= low_stock_alert</code>	Requires immediate restocking attention.
Out of Stock Products	<code>variants WHERE stock_qty = 0</code>	Products blocking parent purchases.
Pending Admin Approvals	<code>products WHERE vendor_id = ? AND approval_status = 'pending'</code>	Products awaiting platform admin review.
Recent Orders	<code>orders WHERE vendor_id = ? ORDER BY created_at DESC LIMIT 10</code>	Last 10 orders with status and total.
Commission Summary	<code>SUM(order_total * commission_rate / 100) FROM orders</code>	Total platform commission deducted this month.
Net Earnings	<code>SUM(order_total) - commission WHERE payment_status = 'paid'</code>	Net amount receivable from platform.

Section 9 — Admin Dashboard Requirements

9. Admin Dashboard Requirements

Widget	Data Source	Business Value
Total Vendors	<code>COUNT(*) FROM vendors BY status</code>	Active, pending, suspended vendor counts.
Pending Product Approvals	<code>COUNT(*) WHERE approval_status = 'pending'</code>	Requires immediate admin attention.
Total Schools	<code>COUNT(*) FROM schools WHERE status = 'active'</code>	Active school count on platform.
Total Parents	<code>COUNT(*) FROM parents</code>	Registered parent/consumer base size.
Orders Overview	<code>COUNT, SUM(order_total) GROUP BY order_status</code>	Placed, confirmed, shipped, delivered orders.
GMV (Gross Merch Value)	<code>SUM(order_total) WHERE payment_status = 'paid'</code>	Total paid order value across all vendors.
Platform Revenue	<code>SUM(order_total * commission_rate/100)</code>	Total commission earned by platform.
Product Analytics	Top products by order volume, revenue per category	Identifies best-performing product categories.
Vendor Performance	Orders, revenue, avg. fulfilment time per vendor	Identifies high and low performing vendors.
School Adoption Rate	Schools with ≥ 1 approved mapping / total schools	Platform penetration within school network.

Section 10 — Future Expansion Roadmap

10. Future Expansion Roadmap

Phase 1: Marketplace MVP	Target: NOW
• Vendor onboarding and KYC	
Phase 2: Commerce Enhancement	Target: Q3 2025
• Returns and refund management module	
Phase 3: Analytics & Scale	Target: Q1 2026
• Multi-vendor analytics dashboard	
Phase 4: Warehouse Management (Optional)	Target: Q3 2026+
• Platform-managed warehouse and stock	

10.1 Why Warehouse Management Must NOT Be in MVP

Including warehouse management in Phase 1 would increase development time by an estimated 6–9 months, add significant operational complexity, and fundamentally change the business model from a marketplace to a fulfilment company. The core value proposition of this platform is its asset-light marketplace model — zero inventory ownership, zero warehouse costs, and zero logistics risk. Warehouse management is only relevant if the platform eventually decides to compete on fulfilment speed and control. This decision should be revisited at Phase 3 based on actual market demand and vendor performance data.

Section 11 — Final Architecture Recommendation

11. Final Architecture Recommendation

As Enterprise Architects, the following recommendations constitute the minimum viable yet production-grade architecture for Phase 1 of the School Uniform & Supplies Marketplace.

11.1 Database Architecture

Component	Technology	Rationale
Primary Database	MySQL 8.0+ or PostgreSQL 14+	Both support UUID natively, JSON columns for future extensibility, and window functions for analytics queries.
Primary Keys	UUID (CHAR(36) or uuid type)	Prevents sequential ID enumeration, supports horizontal sharding, and enables safe API key exposure.
ORM	Laravel Eloquent with query builder	Use Eloquent for standard CRUD; fall back to DB::select() for complex reporting queries to avoid N+1 issues.
Cache Layer	Redis	Cache school product catalogue per school_id+class+gender+year key. TTL = 30 minutes. Invalidate on mapping update.
Search	Laravel Scout + Meilisearch	For product search by name, category, school. Enables typo-tolerant, fast full-text search without MySQL LIKE overhead.
File Storage	AWS S3 + CloudFront CDN	All product images stored in S3. CloudFront CDN for low-latency image delivery across India.
Queue	Laravel Horizon + Redis queues	Process stock decrement, order notifications, and commission calculations asynchronously.

11.2 Index Recommendations

Table	Index Column(s)	Query Pattern
vendors	status	Filter active vendors for listing pages.
products	vendor_id, approval_status	Vendor product listing + admin approval queue.
products	category_id, gender_type	Category and gender filtering on browse.
product_variants	product_id	Load all variants of a product efficiently.
product_variants	stock_qty, low_stock_alert	Low stock and out-of-stock dashboard queries.
product_variants	sku (UNIQUE)	SKU-based product lookup at order time.

school_product_mappings	school_id, class_name, gender_type	Composite index for parent catalogue query.
school_product_mappings	school_id, academic_year	Year-scoped mapping retrieval.
orders	parent_id, order_status	Parent order history with status filter.
orders	vendor_id, created_at	Vendor order inbox with date sort.
order_items	order_id	Fast order detail expansion.

11.3 Scalability Analysis

Horizontal Scaling: UUID PKs and stateless API design allow multiple app server instances behind a load balancer without session affinity. Laravel Sanctum token-based auth supports this natively.

Read Replicas: High-read operations (catalogue browse, product search) should be routed to read replicas. Write operations (orders, stock decrement) must target the primary instance.

Stock Concurrency: Stock decrement must use database-level row locking (SELECT FOR UPDATE) or optimistic locking (version column + check on update) to prevent overselling under concurrent load.

Cache Strategy: School product catalogues are read-heavy and change infrequently. Redis caching with targeted invalidation on mapping/product changes reduces database read load by 60–80%.

Image CDN: Product images served via CloudFront CDN eliminate origin server load. All image paths stored as relative S3 keys, not full URLs, to support CDN domain changes.

Queue Isolation: Stock decrements, commission calculations, and notification dispatches run via dedicated queue workers, isolating write-heavy background work from web request processing.

11.4 Laravel Implementation Guide

Concern	Recommendation
Models	One Eloquent model per table. Use \$fillable and \$casts. Cast UUID fields to string. Cast decimal fields to float. Use SoftDeletes trait on products, variants.
Migrations	UUID PKs via \$table->uuid('id')->primary(). Foreign keys via \$table->foreignUuid(). Decimal via \$table->decimal('mrp', 10, 2).
Policies	Vendor can only CRUD their own products. School can only map platform-approved products. Parents can only view their own orders. Admin has unrestricted access.
Observers	ProductVariant Observer: on updated, if stock_qty < low_stock_alert, dispatch LowStockAlert notification to vendor. On stock_qty = 0, dispatch OutOfStockAlert.
Transactions	Order placement must wrap stock decrement + order creation + order_items creation in a DB::transaction() block. Rollback on any failure.
API Resources	Use Laravel API Resources to transform model data for API responses. Never expose raw Eloquent models. Control field visibility per role.

Events	Dispatch OrderPlaced event on order creation. Listeners handle: StockDecrement, CommissionCalculation, VendorNotification, ParentOrderConfirmation.
Validation	Form Requests per API endpoint. selling_price must be ≤ mrp at validation layer. stock_qty must be non-negative integer.

Document End

School Uniform & Supplies Marketplace — Software Architecture Specification v1.0

Generated on June 14, 2026 | Confidential

Prepared By	Reviewed By	Approved By
Solution Architect	Product Manager	CTO / Technical Lead